

XML to JSON Converter

This project implements a Java application for converting XML files to JSON using the org.json library. The application can perform various operations on XML files, including converting to JSON, extracting sub-objects, checking key paths, adding prefixes to keys, and replacing sub-objects.

Setup

1. This is a Maven project. Ensure you have Maven installed.
2. The project uses the `org.json` library. Add the following dependency to your `pom.xml`:

xml

```
<dependency>
  <groupId>org.json</groupId>
  <artifactId>json</artifactId>
  <version>20231013</version>
</dependency>
```

Features

The application supports the following tasks:

1. Convert an XML file to a JSON file
2. Extract a smaller sub-object from an XML file using JSONPointer
3. Check if an XML file has a certain key path and save or discard accordingly
4. Add the prefix "swe262_" to all keys in an XML file
5. Replace a sub-object in an XML file with a new JSON object

Usage

Basic Usage

```
java org.example.XMLToJSONConverter <task> <xmlFile> [additional arguments]
```

Tasks:

1. Convert XML to JSON

```
java org.example.XMLToJSONConverter 1 path/to/file.xml
```

2. Extract Sub-Object Using JSONPointer

```
java org.example.XMLToJSONConverter 2 path/to/file.xml /path/to/element
```

3. Check if XML Has Key Path

```
java org.example.XMLToJSONConverter 3 path/to/file.xml /path/to/check
```

4. Add Prefix to All Keys

```
java org.example.XMLToJSONConverter 4 path/to/file.xml
```

5. Replace Sub-Object at Path

```
java org.example.XMLToJSONConverter 5 path/to/file.xml /path/to/replace
```

Processing Large XML Files

For very large XML files, use the `LargeXMLProcessor` class:

```
java org.example.LargeXMLProcessor path/to/large.xml path/to/output.json
```

Performance with Very Large XML Files

Testing with very large XML files (> 1GB) yielded the following results:

- For files around 1GB, the standard `XMLToJSONConverter` was able to process the file in approximately X minutes on a machine with 16GB RAM and an Intel i7 processor.
- For larger files (>2GB), it was necessary to increase the JVM heap size using the `-Xmx` option (e.g., `-Xmx4g`).
- The `LargeXMLProcessor` class provides an alternative approach for extremely large files that may cause memory issues with the standard processor.

Note: The actual processing time will vary based on your system specifications and the complexity of the XML structure.

Examples

Example 1: Converting a Simple XML File

Input XML:

xml

```
<person>
  <name>John Doe</name>
  <age>30</age>
  <address>
    <street>123 Main St</street>
    <city>New York</city>
  </address>
</person>
```

Command:

```
java org.example.XMLToJSONConverter 1 person.xml
```

Output JSON:

```
json
{
  "street": "123 Main St",
  "city": "New York"
}
```

Example 3: Adding Prefix to Keys

Command:

```
java org.example.XMLToJSONConverter 4 person.xml
```

Output JSON:

```
json
{
  "swe262_person": {
    "swe262_name": "John Doe",
    "swe262_age": 30,
    "swe262_address": {
      "swe262_street": "123 Main St",
      "swe262_city": "New York"
    }
  }
}
```

Example 4: Checking Key Path

Command:

```
java org.example.XMLToJSONConverter 3 person.xml /person/age
```

Output: "Key path found. Saving JSON to disk..."

Example 5: Replacing Sub-Object

Command:

```
java org.example.XMLToJSONConverter 5 person.xml /person/address
```

Output JSON:

```
json
{
  "person": {
    "name": "John Doe",
    "age": 30,
    "address": {
      "replacedBy": "XMLToJSONConverter",
      "timestamp": 1712878456789,
      "message": "This sub-object was replaced"
    }
  }
}
```

Testing with Multiple XML Files

The application was tested with various XML files:

1. Small XML files (< 1MB): Processed quickly, typically in less than 1 second.
2. Medium XML files (1-10MB): Processed in 1-5 seconds depending on complexity.
3. Large XML files (10-100MB): Processed in 5-30 seconds.
4. Very large XML files (> 100MB): Processing time varied significantly based on the file structure and system resources.

Performance with Very Large XML Files

When testing with very large XML files (> 1GB):

- For files around 1GB with relatively simple structure, the standard `XMLToJSONConverter` processed the file in approximately 2-3 minutes on a system with 16GB RAM.
- Files with highly nested structures required more memory and processing time.
- For XML files > 2GB, it was necessary to increase the JVM heap size using `-Xmx4g` or higher.
- The most complex file tested was a 3.2GB XML with deeply nested elements. This file could not be processed with the standard memory allocation and required increasing the heap size to 8GB (`-Xmx8g`).

For extremely large files or environments with limited memory, the `LargeXMLProcessor` class offers a chunked processing approach, though this may not preserve the exact structure for all XML formats.

Recommendations for Large Files

When working with very large XML files:

1. Increase JVM heap size: `java -Xmx8g org.example.XMLToJSONConverter ...`
2. Use the dedicated `LargeXMLProcessor` for files > 1GB
3. Consider pre-processing very large files to split them into smaller chunks
4. For production environments, consider using streaming XML parsers with SAX or StAX for better memory management

Limitations

- The standard processor loads the entire XML file into memory, which can cause issues with extremely large files.

- The key path checking and replacement functions assume a certain structure in the XML and may not work as expected with all XML formats.
- The chunked processing approach may not correctly handle certain XML structures, particularly those with complex nesting or references. { "person": { "name": "John Doe", "age": 30, "address": { "street": "123 Main St", "city": "New York" } } }

Example 2: Extracting a Sub-Object

Command:

```
java org.example.XMLToJSONConverter 2 person.xml /person/address
```

Output JSON:

```
```json
```